

Adatbázis-kezelés jegyzet

II. Relációs adatmodell

Összeállította: Faludi Anita
2013.

Tartalom

Tartalom	2
Bevezetés	3
A relációs modell, előnyei, főbb jellemzői	4
A relációs adatmodell definíciója	5
A reláció definíciója.....	5
Reláció, tábla, adatbázis.....	6
Egyed típus, reláció.....	8
Összefoglaló kérdések	8
Relációs adatbázis belső szerkezete	9
Kulcs	9
Integritás	10
Funkcionális függőségek	13
Összefoglaló kérdések	15
Normalizálás	15
Anomáliák az adatbázisban.....	15
Az adatbázis hatékonysága	16
Első normál formára (1NF-re) hozás	16
Második normál formára (2NF-re) hozás	18
Harmadik normál formára (3NF-re) hozás	20
Összefoglaló kérdések	22
Műveletek, relációs algebra	22
Átnevezés	23
Korlátozás/kiválasztás (szelekció)	24
Vetület (projekció)	25
Összekapcsolás	26
Keresztszorzat	27
Egyesítés (unió)	28
Metszet.....	29
Különbség	30
Osztas	31
Kiterjesztés.....	32
Csoportosítás.....	32
Források	33

Bevezetés

Ez a jegyzet elsősorban azoknak a diákoknak készült, akiket tanítok, ezért a jegyzet erőteljesen hiányos. Az olvasó egy percig se gondolja azt, hogy a témakörhöz ennyi információ tartozik. A jegyzetben csak azokat a területeket érintettem, amit szükségesnek ítéltem meg, és amiről úgy gondoltam, hogy megfelelő alapot biztosít a továbbfejlődéshez.

A relációs modell, előnyei, főbb jellemzői

A relációs adatmodell napjaink legelterjedtebb adatmodellje; a modellel egyszerű, könnyen megtanulható leírási módot sikerült megvalósítani. Egyszerűségének következtében a felhasználók körében gyorsan népszerűvé vált és a személyi számítógépek piacán sok implementációja született. Az elméleti megalapozottság a kutatók, szakemberek szimpátiáját is kiváltotta, így ez a modell számos új fejlesztés alapját képezi. Az adatmodell fontos előnye az egyszerűség mellett a modell rugalmassága.

Egy egyszerű és könnyen megérthető strukturális részt tartalmaz. A strukturális rész egyszerűsége a közérthetőség mellett az alkalmazhatóságot is növeli, hiszen az egyszerűbb formulák általánosabban fordulnak elő a különböző adatforrásokban.

A modellhez olyan műveleti rész csatlakozik, amely a programozási nyelveknél egyszerűbb kezelői felületet biztosít, annak érdekében, hogy minél általánosabban, minél szélesebb körben lehessen használni.

Az adatmodell integritási részében egyszerű, közérthető és ezzel együtt hatékony feltételeket definiál. Az integritási feltételek előnye, hogy deklaratív megadásukkal a szabályok magában az adatbázisban kerülnek letárolásra, és a relációs adatbázis-kezelő automatikusan ellenőrizni fogja a szabályok betartását, illetve csak olyan tevékenységeket fog megengedni, amelyek nem sértik a megadott integritási előírásokat.

Egyszerű tervezési metodika; az adatbázis tervezése jól definiált, egyértelmű elméleti alapokon nyugszik. A tervezés elméleti alapjai az úgynevezett normalizálási szabályokon alapulnak, amely a relációs modell strukturális lehetőségeit és a modellezendő fogalmak közötti függőségi kapcsolatokat veszi figyelembe. A normalizálás segítségével egy hatékony, áttekinthető adatmodellt kapunk.

Nagyfokú logikai függetlenség. A relációs modell elrejtí a felhasználók elől a megvalósítás részleteit. A felhasználó nyugodtan használhatja a hozzá közel álló fogalmakat, a rendszer egy belső optimalizáló komponens segítségével ki fogja választani a leghatékonyabb fizikai megvalósítást.

A relációs modell tiszta elméleti alapokon nyugszik. A halmazelméletre és a matematikai logikára alapozva a relációs modell az első, mélyebb elméletre támaszkodó adatmodell. Ennek fő előnye, hogy a modell megbízhatóbb és kiszámíthatóbb lesz, és könnyebb a modell tulajdonságainak a vizsgálata is. A megbízhatóság garantálja, hogy nem fogják kellemetlen események, váratlan hibák megzavarni az adatmodellen alapuló adatbázis-kezelők működését.

Egységesség. Az adatbázisban mind a normál, mind a struktúrát leíró metaadatok tárolása ugyanazon módon és formalizmussal történik és ugyanazon utasításokkal valósítható meg.

A relációs adatmodell definíciója

A relációs adatbázisnak több definíciója is létezik. Bár maga a reláció elnevezés a halmazelméletben definiált reláció fogalmára vezethető vissza, a halmazelméleti direkt szorzattal itt most mégsem foglalkozom igazán, az érdeklődők a matematika könyvekben megtalálhatják. Jelen jegyzetben csak annyit érintek a témakörből, amennyit szükségesnek érzek valamilyen magyarázat szempontjából, esetleg a jelrendszerből merítek.

A reláció definíciója

Legyenek adottak az alábbi halmazok:

Vezetéknév = {Kis, Nagy, Kovács, Takács, ...}

Keresztnév = {Zoltán, Anna, Béla, Sándor, ...}

Dátum = {01/01/00, 03/01/27, 22/12/88, 18/01/87, ...}

Település = {Battonya, Budapest, Pécs, Sarkad, ...}

ahol

a **Vezetéknév halmaz** a Magyarországon elfogadott összes vezetéknévet tartalmazza,

a **Keresztnév halmaz** a Magyarországon elfogadott összes keresztnévet tartalmazza,

a **Dátum halmaz** 1900 január 1-től 2000-ig az összes dátumot tartalmazza,

a **Település halmaz** az összes Magyarországi település nevét tartalmazza.

Vegyünk minden halmazból egy-egy elemet, és írjuk ezeket egymás mellé mindig ugyanabban a sorrendben (vezetéknév, keresztnév, dátum, település). Például:

Kis	Sándor	01/01/00	Sarkad
Takács	Anna	18/01/21	Pécs
Takács	Péter	03/01/26	Battonya
Nagy	Gábor	10/12/76	Budapest

és így tovább

Ha az összes halmazból éppen azokat választjuk ki, amelyeknek a négy elemből álló csoportja valamilyen módon összefügg (pl. egy vállalat dolgozójának adatai), akkor elmondhatjuk, hogy egy **relációt**, **ill. relációs modellt hoztunk létre**. Adjunk nevet a relációnak, legyen mondjuk „Nyilvántartás”. Ha az adatok fölé oszlopneveket írunk (jelen esetben a halmazok neveit), akkor táblázathoz jutunk.

Nyilvántartás

Vezetéknév	Keresztnév	Dátum	Település
Kis	Sándor	01/01/00	Sarkad
Takács	Anna	18/01/21	Pécs
Takács	Péter	03/01/26	Battonya
Nagy	Gábor	10/12/76	Budapest

Mint látható, **a reláció tehát egy táblázat**.

II. Relációs adatmodell

Ha az adott halmazokból az összes lehetséges módon felírnánk sorban a négy elemet, akkor a halmazok direkt szorzatát kapnánk. Ennek egy részhalmazát hívjuk **reláció**nak. A reláció összetartozó elemeit (vagyis a táblázat egy sorát) **rekord**nak nevezzük. Tulajdonképpen mondhatjuk, hogy a reláció rekordok halmaza.

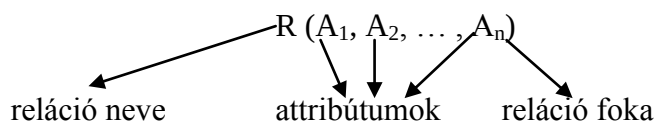
A fenti táblázat utolsó két oszlopa valóban az oszlopnevekhez tartozó értékeket tartalmazza, de nem eldönthető, hogy a dátum és a település mire vonatkozik. Születési dátum és hely, esetleg költözési dátum és új lakhely? Ezeket az elnevezéseket pontosítanunk kell. Tehát az adatbázis egy értelmezett (megfelelő névvel ellátott) tartományát, oszlopát **attribútum**nak (vagy tulajdonságnak), több adatbáziskezelő rendszerben pedig **mező**nek nevezzük.

Nyilvántartás

Vezetéknév	Keresztnév	Születésidátum	Lakóhely
Kis	Sándor	01/01/00	Sarkad
Takács	Anna	18/01/21	Pécs
Takács	Péter	03/01/26	Battonya
Nagy	Gábor	10/12/76	Budapest

Egyértelmű attribútumokat foglalmaztunk meg a relációhoz.

Számontartjuk még a **reláció fokát**, amelyet az attribútumok száma, és a **reláció számosságát**, amelyet pedig a rekordok száma adja meg. Egy **reláció sémájának** jelölése:



A fenti táblázatunkat ennek megfelelően a következő képpen jelöljük:



Reláció, tábla, adatbázis

Egy reláció egyben lehet egy relációs adatbázis is, de általában a relációs adatbázisok több táblából állnak. Azt, hogy egy adatbázist hány táblára bontunk szét, vagy hány táblát foglalunk egy adatbázisba, már a megtervezéskor eldöntjük. Természetesen egy adatbázis általában nem lehet egymástól független táblák halmaza, hiszen így semmit nem érnének velük.

Példa:

DOLGOZÓ			PRÉMIUM	
AZONOSÍTÓ	NÉV	FIZETÉS	ÖSSZEG	DÁTUM
001	Nagy Pál	136.500	18.500	02-03-95
006	Kis Pál	145.900	17.640	05-06-95
009	Közepes Pál	176.512	64.512	12-08-95
			15.600	24-09-95
			62.450	18-12-95

A két tábla között nincs semmilyen kapcsolat, a második tábla ráadásul semmit sem mond. Valószínűleg arról van szó, hogy a három dolgozó közül valaki többször kapott prémiumot, de az is lehet hogy egyetlen dolgozó kapta az összeset. Kellene egy olyan oszlop a második táblázatba, amelyik alapján beazonosítható lenne, hogy melyik prémiumok ki kapta. Bővítsük ki a táblázatot a dolgozók azonosítójával.

PRÉMIUM		
AZONOSÍTÓ	ÖSSZEG	DÁTUM
009	18.500	02-03-95
006	17.640	05-06-95
009	64.512	12-08-95
001	15.600	24-09-95
001	62.450	18-12-95

A két tábla között egy jól felhasználható kapcsolatot építettünk ki. Most már akár fel lehet tenni a két táblára vonatkozóan például olyan kérdést, hogy melyik dolgozó mennyi prémiumot kapott.

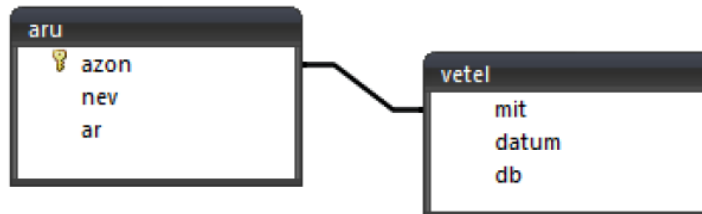
DOLGOZÓ		
AZONOSÍTÓ	NÉV	FIZETÉS
001	Nagy Pál	136.500
006	Kis Pál	145.900
009	Közepes Pál	176.512

PRÉMIUM		
AZONOSÍTÓ	ÖSSZEG	DÁTUM
009	18.500	02-03-95
006	17.640	05-06-95
009	64.512	12-08-95
001	15.600	24-09-95
001	62.450	18-12-95

II. Relációs adatmodell

A relációs adatbázisokban a különböző táblákat **közös attribútumokkal** (oszlopokkal) kötjük össze. Az ilyen oszlopokat **kulcsoszlop**oknak (kulcsmezőnek) is szoktuk nevezni.

DOLGOZÓ (AZONOSÍTÓ, NÉV, FIZETÉS)
↑
↓ kulcsoszlop
PRÉMIUM (AZONOSÍTÓ, ÖSSZEG, DÁTUM)



Kapcsolat megjelenítése Access 2007-ben

Egyed típus, reláció

Korábban az egyed típust a következő képpen adtuk meg:

- **Egyed - egyed típus:** minden olyan dolgot egyednek nevezünk, melynek jellegzetes tulajdonságai vannak. Ezen jellegzetes tulajdonságoknak lehetővé kell tenniük az egyedek megkülönböztetését. Az így megkülönböztetett dolgokról, most már egyedekről adatot, adatokat tárolunk. Egyszerűbben, minden egyed, amiről adatot tudunk tárolni.

Ha a fenti relációinkat megnézzük, akkor pontosan illik rá a definíció. Vagyis elmondhatjuk, hogy egy egyed típus éppen egy reláció, és minden reláció egy egyed(típus)nak tekinthető:

Táblázat = Reláció = Egyed típus.

Összefoglaló kérdések

- Miért hívjuk a relációs adatbázist így?
- A relációs adatmodell: egy táblázat. Mik a tábla oszlopai?
- Mi a reláció foka?
- Mi a reláció számossága?
- Mi „fűzi össze” a relációkat egy adatbázisban?
- Milyen kapcsolat van egy relációs és egy egyed típus között?

Relációs adatbázis belső szerkezete

Kulcs

A kulcs olyan tulajdonság(ok), attribútum(ok), amely(ek) egyértelműen beazonosít(anak) egy rekordot. Sokféle jelölési módja van, én a folyamatos aláhúzást választottam: KULCS

- **Kulcsmező, kulcsoszlop**

A relációs adatbázisokban a különböző táblákat összekötő attribútum.

- **Egyszerű kulcs**

Egy megadott attribútum önmagában elég egy rekord beazonosításához.

Pl. személyi szám, tanulói azonosító, rendszám, stb.

- **Összetett kulcs**

DEF: Több attribútum szükséges egy rekord beazonosításához.

Megjegyzés: Egy relációban minden esetben van kulcs akkor is, ha külön nem jelölünk meg egyet sem. Mivel nem megengedett két teljesen azonos rekord, ezért a legrosszabb esetben az összes tulajdonságot meg kell vizsgálni az azonosításhoz.

- **Elsődleges kulcs**

DEF: A lehetséges egyedi tulajdonságok közül az, amelyet azonosítóként megjelölünk.

Megjegyzés: Előfordulhat, hogy több olyan attribútum fordul elő a relációban, amely önmagában azonosításra alkalmas, pl. személyi szám és a személyi igazolvány szám is szerepel. Ebben az esetben meg kell adni, hogy melyiket szeretnénk azonosításra használni.

- **Külső kulcs, idegen kulcs**

DEF: A reláció azon attribútuma, amely egy másik relációban elsődleges kulcs. Sokféle jelölési módja van, én a szaggatott aláhúzást választottam: Külső KULCS

Megjegyzés: Az idegen kulcs tulajdonképpen a relációban nem azonosít semmit, csak a kapcsolatot fejezi ki.

Megjegyzés: Egy relációban több idegen kulcs is szerepelhet, pl.

DOLGOZO (AZON, NEV, FIZ)

ELADAS (RENDELESSZAM, VEVOKOD, AZON)

VEVO (VEVOKOD, VEVONEV, CÍM)

Az ELADAS reláció egy elsődleges és két idegen kulcsot tartalmaz.



Kapcsolat megjelenítése Access 2007-ben

Integritás

Adatintegritás → adatérvényesség

- **Alapreláció** (fő reláció)

DEF: Az alaprelációnak annak érdekében, hogy a reláció sorai különböző elemek halmazát alkossák, mindegyik sorban külön azonosítóval kell rendelkeznie.

különböző értékek →

AZONOSÍTÓ	NÉV	FIZETÉS
001	Nagy Pál	136.500
006	Kis Pál	145.900
009	Közepes Pál	176.512

← *alapreláció*

idegen kulcs →

AZONOSÍTÓ	ÖSSZEG	DÁTUM
009	18.500	02-03-95
006	17.640	05-06-95
009	64.512	12-08-95
001	15.600	24-09-95
001	62.450	18-12-95

- **Egyedintegritás**

Az egyedintegritás megköveteli, hogy az elsődleges kulcsot alkotó attribútumok egyike se kaphasson NULL értéket.

ilyen nem fordulhat elő →

AZONOSÍTÓ	NÉV	FIZETÉS
001	Nagy Pál	136.500
	Kis Pál	145.900
009	Közepes Pál	176.512

- **Hivatkozási integritás**

DEF: Az idegen kulcsokban lévő minden nem NULL értéknek léteznie kell abban a relációban, amelyikben az elsődleges kulcs része.

DOLGOZÓ

AZONOSÍTÓ	NÉV	FIZETÉS
001	Nagy Pál	136.500
009	Közepes Pál	176.512

ez az érték itt
nem szerepelhet,
mert az
alaprelációban
sem szerepel

PRÉMIUM

AZONOSÍTÓ	ÖSSZEG	DÁTUM
009	18.500	02-03-95
006	17.640	05-06-95
009	64.512	12-08-95
001	15.600	24-09-95
001	62.450	18-12-95

Probléma: hogyan tudjuk módosítani a fő relációban lévő kulcsot?

Megoldási lehetőségek:

1. Korlátozás
 - Nem engedem módosítani
2. Kaszkádolás
 - Ha törölöm az alaprelációban lévő kulcsmező értékét, az összes olyan rekord törlődik, amely kapcsolatban állt vele.

DOLGOZÓ

AZONOSÍTÓ	NÉV	FIZETÉS
001	Nagy Pál	136.500
006	Kis Pál	145.900
009	Közepes Pál	176.512

PRÉMIUM

AZONOSÍTÓ	ÖSSZEG	DÁTUM
009	18.500	02-03-95
006	17.640	05-06-95
009	64.512	12-08-95
001	15.600	24-09-95
001	62.450	18-12-95

II. Relációs adatmodell

- Ha módosítom az alaprelációban lévő kulcsmező értékét, akkor az összes olyan mező módosul, amely a módosításra „hivatkozik”

DOLGOZÓ

AZONOSÍTÓ	NÉV	FIZETÉS
007	Nagy Pál	136.500
006	Kis Pál	145.900
009	Közepes Pál	176.512

PRÉMIUM

AZONOSÍTÓ	ÖSSZEG	DÁTUM
009	18.500	02-03-95
006	17.640	05-06-95
009	64.512	12-08-95
007	15.600	24-09-95
007	62.450	18-12-95

3. Nullázás

- Az alaprelációban történt törlés vagy módosítás esetén, az össze erre hivatkozó kulcsmező értéke NULL lesz.

DOLGOZÓ

AZONOSÍTÓ	NÉV	FIZETÉS
007	Nagy Pál	136.500
006	Kis Pál	145.900
009	Közepes Pál	176.512

PRÉMIUM

AZONOSÍTÓ	ÖSSZEG	DÁTUM
006	18.500	02-03-95
	17.640	05-06-95
	64.512	12-08-95
	15.600	24-09-95
	62.450	18-12-95

Vagyis a hivatkozási integritás megköveteli, hogy minden idegen kulcsértéknek hivatkoznia kell egy létező elsődleges kulcsértékre, ellenkező esetben fel kell vennie a NULL értéket.

Funkcionális függőségek

DEF: A funkcionális függőség a reláció attribútumai között lévő szemantikai kapcsolat, vagyis az attribútumok egy csoportja függ az attribútumok egy másik csoportjától.

ÜGYFÉL

ÜAZON	NÉV	HELYSÉG	STÁTUSZ	SZLASZÁM	EGYENLEG
2345	Kiss Pista	Budapest	üzleti	120768	237
2345	Kiss Pista	Budapest	üzleti	348973	-456
7654	Nagy Géza	Vác	magán	987654	12 567
8765	Közepes Luca	Győr	üzleti	745363	789
8765	Közepes Luca	Győr	magán	678453	-23
8765	Közepes Luca	Győr	üzleti	348973	-456

A fenti reláció egy banki adatbázisba nyújt betekintést. A reláció sémája:

ÜGYFÉL (ÜAZON, NÉV, HELYSÉG, STÁTUSZ, SZLASZÁM, EGYENLEG)

ahol

ÜAZON Az ügyfél azonosító száma

NÉV Az ügyfél neve

HELYSÉG Az ügyfél értesítési címe

STÁTUSZ Milyen minőségben tulajdonosa a számlának

SZLASZÁM A számla száma

EGYENLEG Az aktuális egyenleg ezer forintban.

A fenti reláció néhány függősége:

$\{\text{ÜAZON}\} \rightarrow \{\text{NÉV}\}$

Az ügyfél azonosítójából megtudhatjuk az ügyfél nevét. Fordítva már ez nem igaz, hiszen ugyanolyan nevű ügyfelek előfordulhatnak.

$\{\text{ÜAZON}, \text{SZLASZÁM}\} \rightarrow \{\text{EGYENLEG}\}$

Látható, hogy az ügyfél azonosítója kevés az aktuális egyenleg megállapításához, hiszen egy ügyfélhez több számla, ebből következőleg számlaszám is tartozhat. De a két attribútum együttesen már meg tudja határozni az egyenleget.

$\{\text{SZLASZÁM}\} \rightarrow \{\text{EGYENLEG}\}$

Azt is észre kell ugyanakkor venni, hogy az egyenleg a számlaszámtól külön is függ

$\{\text{SZLASZÁM}\} \rightarrow \{\text{STÁTUSZ}\}$

De látható, hogy a státusz is függ a számlaszámtól. Az előző két függőségeket rövidebben is leírhatjuk:

$\{\text{SZLASZÁM}\} \rightarrow \{\text{EGYENLEG}, \text{STÁTUSZ}\}$

Megállapíthatjuk, hogy egy relációhoz sok függőség tartozhat. És itt említenék meg két speciális függőséget.

- **Tranzitív függőség**

DEF: Két funkcionálisan függő attribútumhalmaz között található egy harmadik, amely úgy mond „átviszi” a függőséget.

Adott a következő relációs séma:

DOLGOZÓ(SZEMÉLYISZÁM, HAVIBÉR, ÉVESBÉR)

Tudjuk továbbá, hogy az ÉVESBÉR a HAVIBÉR 12-szerese.

Kulcs a SZEMÉLYISZÁM.

Ezek figyelembe vételével nyilvánvaló, hogy fennállnak az alábbi függőségek:

$\{SZEMÉLYISZÁM\} \rightarrow \{HAVIBÉR\}$

$\{SZEMÉLYISZÁM\} \rightarrow \{ÉVESBÉR\}$

Ugyanakkor

$\{HAVIBÉR\} \rightarrow \{ÉVESBÉR\}$

függőség is teljesül. Tehát a fenti példában létezik tranzitív függőség. Jelölése:

$\{SZEMÉLYISZÁM\}$
 $\swarrow \quad \searrow$
 $\{HAVIBÉR\} \rightarrow \{ÉVESBÉR\}$

- **Teljes függőség**

DEF: Egy attribútumhalmaz teljesen függ egy másik attribútumhalmaztól, ha nem függ külön az attribútumhalmaz egyik részalmazától sem.

Adott az alábbi reláció

ELADÁS		
DÁT	TKÓD	DB
02/03	TX-5	2
02/03	XB-3	6
02/04	ET-2	4
02/04	FG-G	5
02/04	TX-5	6
02/05	XB-3	7

ahol

DÁT az eladás napja

TKÓD az eladott termék kódja

DB az adott kódú termékből eladott darabszám.

A $\{DÁT, TKÓD\} \rightarrow \{DB\}$ függőség láthatóan létezik. De az is belátható, hogy a DB attribútum külön a dátumtól nem függ, mert egy nap több termékből is eladhatnak ugyanannyi darabot, és külön a termékkódtól sem függ, mert más-más napokon is eladhatnak ugyanolyan terméket. Vagyis a fent megállapított függőség teljes.

Összefoglaló kérdések

- Milyen attribútumok alkotnak kulcsot?
- Mit nevezünk elsődleges kulcsnak?
- Mi a külső kulcs?
- Mi az alapreláció lényege?
- Mit nevezünk egyedintegritásnak?
- Mit nevezünk hivatkozási integritásnak?
- Milyen problémákat vet fel a hivatkozási integritás?
- Mit jelent a kaszkádolás?
- Definiálja a funkcionális függőséget!
- Mikor mondjuk, hogy egy függőség teljes?
- Mikor mondjuk, hogy egy függőség tranzitív?

Normalizálás

Anomáliák az adatbázisban

Egy adatbázisrendszer felé alapkövetelmény a biztonságos adattárolás, gyors és pontos keresés, a naprakészség. Egy rosszul megtervezett adatbázis rosszul felépülő logikai kapcsolatokat eredményez, amellyel veszélybe kerül az adatbiztonság és ez különböző anomáliákhoz (hibához, rendellenességhez) vezethet.

- **Redundancia**

A redundancia adatismétlődést jelent, amely többféle lehet

→ Káros redundancia

Adatok felesleges ismétlődése, tárolása, amelynek eredménye:
többletmunkával jár,
lelassul a munkamenet,
megnő a hibázás lehetősége,
megnő az adatbázis mérete.

→ Ellenőrzött redundancia

Szándékosan, a tervező által elhelyezett adatismétlődés, ami általában számított mező, ami gyorsabbá teheti a műveletvégzést.

→ Optimális redundancia

Ez biztosítja az adatbázisban a relációk közötti kapcsolatokat. Ugyanaz az adat egyik relációban kulcs, másikban pedig idegen kulcs.

- **Módosítási anomália**

Káros redundancia esetén fordul elő, amikor minden rekordot végig kell vizsgálni ahhoz, hogy hibamentesen lehessen az adatbázist módosítani.

- **Törlési anomália**

Akkor lép fel, amikor csak adatvesztés árán lehet kitörölni valamit az adatbázisból.

- **Bővítési anomália**

Olyankor fordul elő, amikor valamilyen szerkezeti okból nem engedélyezett az adatfelvitel

Az adatbázis hatékonysága

Egy adatbázis akkor tekinthető hatékonynak, ha

- Nem tartalmaz káros (felesleges) redundanciát,
- Kerüli, illetve minimális mértékben használ NULL értéket,
- Nem fordulnak elő anomáliák.

- **Normalizáció**

DEF: A normalizáció, vagy normál formára hozás az a folyamat, amellyel a relációs adatbázis struktúrájának hatékonyságát biztosítjuk.

Első normál formára (1NF-re) hozás

- **1NF**

DEF: Egy **R** relációról azt mondjuk, hogy első normál formában, vagyis 1NF-ben van, ha minden sorában pontosan egy attribútumérték szerepel.

Vizsgáljuk meg az alábbi relációkat első normál forma szempontjából:

1. tábla

VASARLASOK						
AZON	DATUM	CIKKSZAM	EGYSEG	VEVOKOD	VEVO	DARAB
1	05.08.	3	db	2	Péter	2
2	06.02.	1	db	3	Pál	10
		2	doboz			5
3	06.05.	2	doboz	1	Pista	8
4	07.09.	1	db	2	Péter	3
		3	db			1

Többértékű attribútumok: CIKKSZAM, EGYSEG, DARAB

2. tábla

DOLGOZOK				
AZON	VEZETEKNEV	KERESZTNEV	SZAKKÉPZETTSÉG	SZULDATUM
001	Nagy	Zsolt	gépészmérnök programozó közgazdász	52/02/16
002	Kis	Pál	zenész	58/08/08
003	Közepes	Péter	lakatos asztalos	65/06/09

Többértékű attribútum: SZAKKÉPZETTSÉG

A definíció szerint egyik táblázat sincs 1NF-ben, a relációs adatbáziskezelő rendszerek sem tudják ilyen formában kezelni az adatokat, ezért át kell alakítani.

- Első módszer**

A legegyszerűbb módszer, hogy a többértékű attribútumokat elemeire bontjuk. Ezáltal a táblánkban ugyan több sor lesz, és az eddig elemi értékek többször szerepelnek majd, de így már megfelel az 1NF-re vonatkozó szabályoknak.

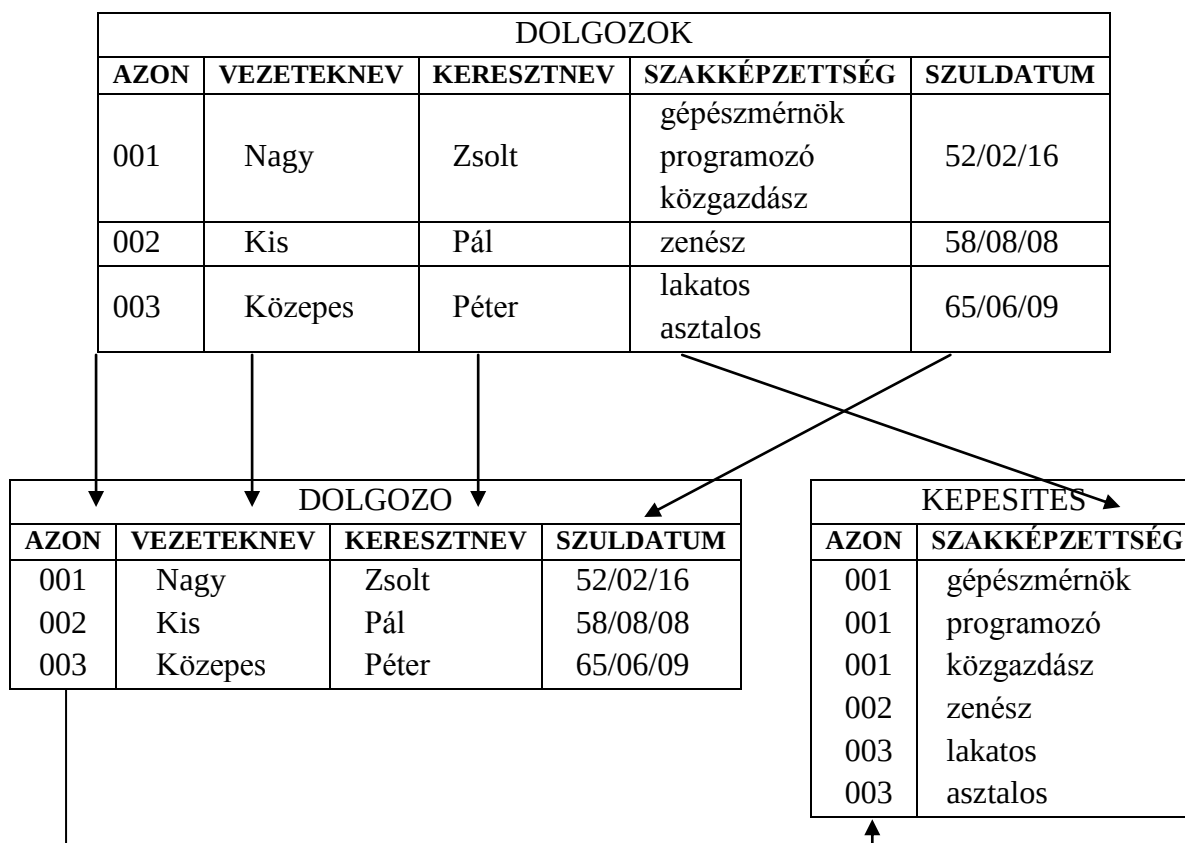
VASARLASOK						
AZON	DATUM	CIKKSZAM	EGYSEG	VEVOKOD	VEVO	DARAB
1	05.08.	3	db	2	Péter	2
2	06.02.	1	db	3	Pál	10
2	06.02.	2	doboz	3	Pál	5
3	06.05.	2	doboz	1	Pista	8
4	07.09.	1	db	2	Péter	3
4	07.09.	3	db	2	Péter	1

elemi értékek többszörözése

többértékű attribútumok elemekre bontása

- Második módszer**

Az eredeti táblánkat két új táblára bontjuk. Az első táblába kerülnek azok az attribútumok, amelyek csak elemi értékeket tartalmaznak, (jelen esetben az AZON, VEZETEKNEV, KERESZTNEV, SZULDATUM). A többszörös attribútumértékekkel rendelkezőket egy másik táblába gyűjtjük (jelen esetben a SZAKKÉPZETSÉG), és külső kulcsot adunk hozzá, amely összeköti az első táblával. Ebben az esetben csak a külső kulcs tartalmaz ismétlődést (optimális redundancia).



Második normál formára (2NF-re) hozás

- 2NF

DEF: Egy R relációról azt mondjuk, hogy második normál formában, vagyis 2NF-ben van, ha

- 1NF-ben van, és
- minden attribútum teljesen függ a kulcstól.

SZERDA			
TEREM	FEROHELY	IDOPONT	ELŐADÁS
23	30	11:00	tőri
		12:00	matek
18	15	12:00	kémia
		13:30	tőri
29	30	11:00	matek
		12:00	kémia

A fenti táblázat nincs 2NF-ben, mert már a definíció szerinti első kritériumnak sem felel meg, vagyis nincs 1NF-ben többértékű attribútumok miatt.

- **Első lépés – hozzuk a táblát 1NF-re**

Most a legegyszerűbb módszert választom az első normál formára hozáshoz:

SZERDA			
TEREM	FEROHELY	IDOPONT	ELŐADÁS
23	30	11:00	töri
23	30	12:00	matek
18	15	12:00	kémia
18	15	13:30	töri
29	30	11:00	matek
29	30	12:00	kémia

A reláció most már 1NF-ben van.

- **Második lépés – vizsgálat: 2NF-ben van?**

A 2NF megállapításához szükségünk van a kulcs ismeretére. A táblához viszont nem adtam meg kulcsot, de a függőségek vizsgálatával ezt megtehetjük. Nézzünk meg a függőségeket, hogy megállapíthassuk a kulcsot.

Függőségek:

$\{TEREM\} \rightarrow \{FEROHELY\}$

A TEREM attribútumtól más már nem függ, így önmagában biztos, hogy nem lehet kulcs. A FEROHELY attribútumtól semmi sem függ, ezért a TEREM-mel együtt sem alkalmas kulcsnak. Keressünk újabb lehetőséget:

$\{TEREM, IDOPONT\} \rightarrow \{FEROHELY\}$

$\{TEREM, IDOPONT\} \rightarrow \{ELOADAS\}$

Megtaláltuk az összetett kulcsunkat, vagyis a reláció sémája a következő:

SZERDA (TEREM, IDOPONT, FEROHELY, ELOADAS)

Vizsgálat:

A definíció szerinti első kritérium teljesült, hisze a táblázatunk 1NF-ben van. A kérdés most már csak az, hogy a kulcstól teljesen függ-e az összes többi attribútum. Vagyis azt kell keresni, hogy van-e olyan tulajdonság a táblázatban, ami önmagában is függ a kulcs valamelyik részétől. A választ rögtön megkapjuk, hiszen az első függőség, amit megállapítottunk: $\{TEREM\} \rightarrow \{FEROHELY\}$. Tehát a reláció nincs 2NF-ben.

- **Harmadik lépés – hozzuk a táblát 2NF-re**

A második normál formára hozás tulajdonképpen a tábla szétszedését jelenti úgy, hogy olyan új táblákat hozunk létre, amelyekre igaz, hogy teljes függőség áll fenn a kulcs és a többi atribútum között.

II. Relációs adatmodell

Módszer:

Felírjuk az összes függőséget. A nem teljes függőségekből alkotunk egy új táblát, majd azt a nem kulcs attribútumot, amelyet már felhasználtunk, kihagyjuk az eredeti táblából jelen esetben:

$\{\text{TEREM}, \text{IDOPONT}\} \rightarrow \{\text{FEROHELY}\}$

$\{\text{TEREM}, \text{IDOPONT}\} \rightarrow \{\text{ELOADAS}\}$

$\{\text{TEREM}\} \rightarrow \{\text{FEROHELY}\}$

HELYSZIN	
TEREM	FEROHELY
23	30
18	15
29	30

SZERDA		
TEREM	IDOPONT	ELŐADÁS
23	11:00	töri
23	12:00	matek
18	12:00	kémia
18	13:30	töri
29	11:00	matek
29	12:00	kémia

Jelen állapotában mindkét relációról elmondható, hogy 2NF-ben van, hiszen megfelel a definícióban megfogalmazott kritériumoknak.

A két reláció sémája:

HELYSZIN (TEREM, FEROHELY)

SZERDA (TEREM, IDOPONT, ELOADAS)

- **Megjegyzés:**

Könnyen belátható, hogy az alábbi esetekben egy reláció 2NF-ben van:

- A táblának egyszerű kulcsa van
- Minden attribútum része a kulcsnak.

Harmadik normál formára (3NF-re) hozás

- **3NF**

DEF: Egy R relációról azt mondjuk, hogy második normál formában, vagyis 3NF-ben van, ha

- 2NF-ben van, és
- az attribútumok között nincs tranzitív függőség

- **Megjegyzés:**

Ebben a fejezetben nem vizsgálok táblákat 1NF-re, mert csak sémákkal dolgozom.

- **Első lépés – vizsgálat 2NF-re**

Adott a következő relációs séma:

VASARLAS (SZAMLASZAM, DATUM, CIKKSZAM, CIKKNEV, EGYSEG, VEVOKOD, VEVONEV, MENNYISEG, FIZETENDO)

A definíció szerint az egyik kritérium, hogy a táblázatunk 2NF-ben legyen, tehát vizsgáljuk meg a táblát teljes függőség szempontjából: van-e olyan attribútum, amely csak az egyik, vagy másik kulcstól függ? Már ránézésre megmondhatjuk, hogy $\{\text{CIKKSZAM}\} \rightarrow \{\text{CIKKNEV}\}$ függőség fennáll, tehát van. Ebből következik, hogy a reláció nincs 2NF-ben.

- **Második lépés – 2NF-re hozás**

Írjuk fel a függőségeket a kulcsokra, és alkalmazzuk a szétbontást:

$$\left\{ \begin{array}{l} \{\text{SZAMLASZAM}, \text{CIKKSZAM}\} \rightarrow \{\text{DATUM}\} \\ \{\text{SZAMLASZAM}, \text{CIKKSZAM}\} \rightarrow \{\text{CIKKNEV}\} \\ \{\text{SZAMLASZAM}, \text{CIKKSZAM}\} \rightarrow \{\text{EGYSEG}\} \\ \{\text{SZAMLASZAM}, \text{CIKKSZAM}\} \rightarrow \{\text{VEVOKOD}\} \\ \{\text{SZAMLASZAM}, \text{CIKKSZAM}\} \rightarrow \{\text{VEVONEV}\} \\ \{\text{SZAMLASZAM}, \text{CIKKSZAM}\} \rightarrow \{\text{MENNYISEG}\} \\ \{\text{SZAMLASZAM}, \text{CIKKSZAM}\} \rightarrow \{\text{FIZETENDO}\} \end{array} \right.$$

$$\left\{ \begin{array}{l} \{\text{SZAMLASZAM}\} \rightarrow \{\text{DATUM}\} \\ \{\text{SZAMLASZAM}\} \rightarrow \{\text{VEVOKOD}\} \\ \{\text{SZAMLASZAM}\} \rightarrow \{\text{VEVONEV}\} \\ \{\text{SZAMLASZAM}\} \rightarrow \{\text{FIZETENDO}\} \end{array} \right.$$

$$\left\{ \begin{array}{l} \{\text{CIKKSZAM}\} \rightarrow \{\text{CIKKNEV}\} \\ \{\text{CIKKSZAM}\} \rightarrow \{\text{EGYSEG}\} \end{array} \right.$$

A 2NF-re hozás szabályait alkalmazva három relációt kapunk, ezek sémái:

VASARLAS (SZAMLASZAM, CIKKSZAM, MENNYISEG)

SZAMLA (SZAMLASZAM, DATUM, VEVOKOD, VEVONEV, FIZETENDO)

CIKKEK (CIKKNEV, EGYSEG)

- **Harmadik lépés – vizsgálat 3NF-re**

A definíció szerint a második kritérium az, hogy a táblázatunkban nem lehet tranzitív függőség, tehát vizsgáljuk meg a relációkat ebből a szempontból: van-e olyan attribútum, amely nem a kulcstól függ?

A VASARLAS táblában nincs tranzitív függőség, mert a kulcsoktól függ a maradék egy attribútum. Hasonlóan a CIKKEK táblában sincs. Ezek 3NF-ben vannak.

A SZAMLA táblában találunk olyan függőséget, amely nem kapcsolódik a kulcshoz: $\{\text{VEVOKOD}\} \rightarrow \{\text{VEVONEV}\}$, tehát van tranzitív függőség. Ezt meg kell szüntetni.

- **Negyedik lépés – 3NF-re hozás**

A tranzitív függőséget a tábla szétszedésével tudjuk megszüntetni. Vizsgáljuk meg megint a függőségeket az adott táblára:

$$\begin{array}{l} \{\text{SZAMLASZAM}\} \rightarrow \{\text{DATUM}\} \\ \{\text{SZAMLASZAM}\} \rightarrow \{\text{VEVOKOD}\} \\ \{\text{SZAMLASZAM}\} \rightarrow \{\text{VEVONEV}\} \\ \{\text{SZAMLASZAM}\} \rightarrow \{\text{FIZETENDO}\} \\ \{\text{VEVOKOD}\} \rightarrow \{\text{VEVONEV}\} \end{array}$$

II. Relációs adatmodell

A tranzitív függőséget okozó attribútumokból új táblát hozunk létre úgy, hogy megjelöljük kulcsnak az arra alkalmasat (amelyiktől függ a másik). Az eredeti táblában pedig csak egy külső kulcsot hagyunk meg tulajdonságok közül. Az így kialakult sémák:

VEVOK (VEVOKOD, VEVONEV)

SZAMLAK (SZAMLASZAM, DATUM, VEVOKOD, FIZETENDO)

- **Végeredmény:**

Az eredeti adatbázist – ami egy táblából állt – hatékonyabbá tettük a normalizálás segítségével, így négy táblát kaptunk:

VASARLAS (SZAMLASZAM, CIKKSZAM, MENNYISEG)

CIKKEK (CIKKNEV, EGYSEG)

VEVOK (VEVOKOD, VEVONEV)

SZAMLAK (SZAMLASZAM, DATUM, VEVOKOD, FIZETENDO)

Összefoglaló kérdések

- Növekszik-e a sorok száma, ha 1NF-re hozunk relációt!
- Lehet-e 2NF-ben egy reláció, ha az nincs 1NF-ben?
- 2NF-ben van-e az adatbázis, ha a kulcs egyszerű?
- A 2NF-re hozásnál mely attribútumhalmazokat kell kiemelni?
- Elveszítünk-e funkcionális függőségeket, amikor 2NF-re hozunk?
- Igaz-e, hogy ha egy reláció 3NF-ben van, akkor 2NF-ben is van? És fordítva?
- Van-e tranzitív függőség a 3NF-es relációkban?
- Meg lehet-e szüntetni a tranzitív függőséget? Hogyan?

Műveletek, relációs algebra

A relációs modell felépítésében annak strukturális és integritási komponensei vesznek részt, e komponensekből áll össze a relációs adatstruktúra. Az itt megadott szabályok határozzák meg, hogy hogyan nézhet ki egy adott adatbázis. Ezek a komponensek az adatbázis bármely időpontbeli állapotaira vonatkoznak, időtől függetlenek, ezért statikus elemeknek nevezzük őket.

Az adatbázisok azonban nem holt, befagyott rendszerek, struktúrák. Az adatbázis értelme, hogy használják azt. A felhasználás során a felhasználók lekérdezhetik, vagy módosíthatják az adatbázis tartalmát. Az adatbázis akkor lesz tehát élő, ha csatlakozik hozzá egy olyan funkciócsoport is, amely lehetővé teszi az adatbázisban tárolt adatok lekérdezését és módosítását. Ezeket a komponenseket – mivel az adatbázis változásaihoz, megváltoztatásához kapcsolódnak – dinamikus komponenseknek nevezzük. A relációs adatmodell műveleti része ezek a dinamikus adatkezelő és adatlekérdező lehetőségeket foglalja magába.

A relációkból elméletileg igen sokféleképpen, többféle mechanizmussal lehet adatokat kiolvasni. Az itt ismertetésre kerülő műveletcsoportot összefoglalóan **relációs algebrának** nevezik. A relációs algebra az **alapja** a ma már szabványként elfogadott és leginkább elterjed adatlekérdező relációs parancsnyelvnek, az **SQL-nek** is.

A **relációs adatmodell műveleti része a relációkon alapul**, vagyis minden művelet a relációkon értelmezett, bemenő operandusai csak relációk lehetnek. A relációs algebra egyszerű, de hatékony módszereket ad a kezünkbe ahhoz, hogy a meglévő relációkból új relációkat hozzunk létre.

A relációs **műveletek csoportjai**:

1. egy művelet, ami nem befolyásolja a reláció sorait, de módosítja a reláció sémáját, vagyis az attribútumok és a reláció nevét – ez az átnevezés,
2. műveletek, amelyek a reláció bizonyos részeit elhagyják – például a korlátozás/kiválasztás, vetítés,
3. műveletek, amelyek két reláció sorait kombinálják – például a keresztszorzat és az összekapcsolás,
4. hagyományos halmazműveletek – például az egyesítés, metszet, és különbség, osztás,
5. egyéb (nem klasszikus értelemben vett relációs algebrai műveletek) – például a kiterjesztés, csoportosítás.

E műveletek nem elegendőek a relációkon elvégzendő összes számításhoz, ugyanis nagyon korlátozott lehetőséget biztosítanak, mégis jelentős részét tartalmazzák azoknak a műveleteknek, amelyeket az adatbázisokkal tenni akarunk.

A műveletek egyik része **adatkezelésre** (felvitel, módosítás, törlés), a másik része pedig **adatlekérdezésre** (adatok különböző szempontok szerinti megjelenítésére) használható.

Tekintsük át a relációs algebra körébe tartozó **műveleteket**:

Egyoperandusú:

- átnevezés ,
- korlátozás (szelekció),
- vetület (projekció)

Kétooperandusú:

- összekapcsolás,
- keresztszorzat,
- egyesítés (unió),
- metszet,
- különbség,
- osztás,

Egyéb:

- kiterjesztés,
- csoportosítás.

Átnevezés

A relációs algebra talán legegyszerűbb művelete. A művelet:

RENAME(oszlop1,oszlop2)

Végrehajtás után az oszlop1 nevű oszlop neve oszlop2 lesz.

II. Relációs adatmodell

Példa: Az Oktatás táblázatban tartjuk nyilván a tanárok nevét és beosztását. Az eredeti relációnk: Oktatás(tanárnév,beosztás). Mivel nem csak tanárok oktatnak, módosítjuk az oszlop nevét: RENAME(tanárnév,oktatónév).

Az átnevezés eredménye: Oktatás(oktatónév,beosztás).

Megvalósítás Acces 2007-ben (tábla szerkesztő nézete)

Oktatás	
Mezőnév	Adattípus
Tanárnév	Szöveg
Beosztás	Szöveg

Korlátozás/kiválasztás (szelekció)

A relációnak azokat a sorait kapjuk eredményül, amelyek megfelelnek egy adott feltételnek.

A művelet:

RESTRICT relációnév WHERE feltétel

Összetett feltételt is megadhatunk az AND (és) OR (vagy) NOT (nem) logikai kifejezések és a $>$ $=$ $<$ relációjelek használatával.

Példa: Az alábbi táblából kiválogatjuk a fehér színű Opel típusú autók adatait:

FehérOpel = RESTRICT Autó WHERE szín="fehér" AND típus="Opel".

Eredményül egyetlen sort kapunk.

Autó			
rendszám	típus	szín	ár
ATC-812	Opel	fehér	3 000 000
RSA-101	Opel	kék	2 800 000
LAK-941	Suzuki	fehér	1 900 000

↓

FehérOpel			
rendszám	típus	szín	ár
ATC-812	Opel	fehér	3 000 000

Megvalósítás Acces 2007-ben (választó lekérdezés)

FehérOpel

Autó

- *
 - rendszám
 - típus
 - szín
 - ár

Mező:	rendszám	típus	szín	ár
Tábla:	Autó	Autó	Autó	Autó
Rendezés:				
Megjelenítés:	☒	☒	☒	☒
Feltétel:		"Opel"	"fehér"	
vagy:				

Vetület (projekció)

A projekció a táblázat leszűkítését jelenti bizonyos oszlopaira. Akkor használjuk, ha csak bizonyos oszlopok tartalmára vagyunk kíváncsiak. A művelet:

relációnév PROJECT(oszlop1,oszlop2...oszlopn)

Példa: Csak az autók típusára vagyunk kíváncsiak:

AutóTípus = Autó PROJECT(típus)

Autó

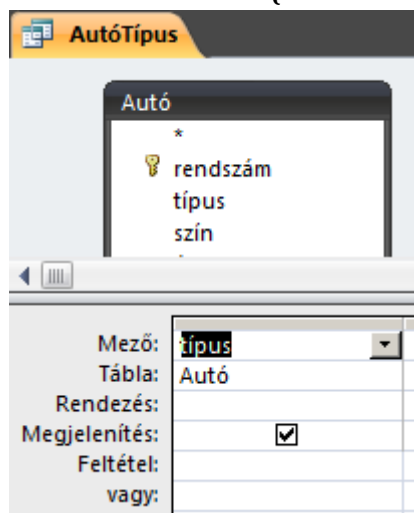
rendszer	típus	szín	ár
ATC-812	Opel	fehér	3 000 000
RSA-101	Opel	kék	2 800 000
LAK-941	Suzuki	fehér	1 900 000

↓

AutóTípus

típus
Opel
Opel
Suzuki

Megvalósítás Acces 2007-ben (választó lekérdezés)



Mint ebben az esetben is, néha előfordulhat, hogy kapott új táblában két, vagy több rekord is meg fog egyezni, márpedig az elméleti relációs adatmodellben ilyen nem lehetséges. A gyakorlatban azonban az adatbázis-kezelő rendszerek minden előfordulást meghagynak az eredménytáblában, mivel a többszöri előfordulás is információt nyújthat az adatbázis felhasználójának. Az adatbázis-kezelők általában külön beállítási lehetőséget biztosítanak a többszörösen előforduló rekordok kiszűrésére.

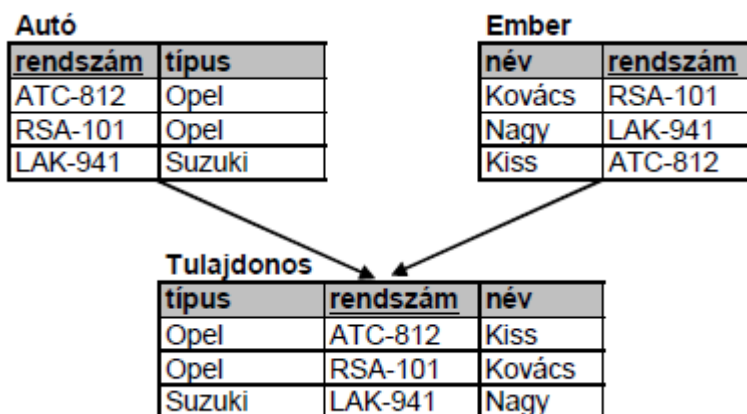
Összekapcsolás

Az összekapcsolás, egyesítés művelete szintén két relációból állít elő egy eredmény relációt. Akkor használjuk, ha egynél több táblázatból kell összeszedni az adatokat, vagyis összerakjuk azt, amit normalizálással szétszedtünk. Az eredmény egy olyan reláció lesz, amelyben az egyik reláció soraihoz hozzáírjuk a másik reláció minden olyan sorát, amelyben a megadott közös mezők (join mezők) értéke azonos. Az új táblázat oszlopainak száma a két kiinduló táblázat oszlopainak összege mínusz a közös oszlopok száma. A közös oszlop csak egyszer szerepel. A művelet:

relációnév1 JOIN relációnév2(join mező)

Példa: Az autók és tulajdonosaik adatait két táblában tároljuk. Létrehozunk egy olyan táblát, ami a tulajdonos adatai mellett az autójának adatait is tartalmazza:

Tulajdonos = Autó JOIN Ember(rendszer)



Keresztszorzat

Két reláció keresztszorzatát úgy kapjuk meg, hogy az első reláció minden sorához hozzáírjuk a második reláció minden sorát. A műveletet abban az esetben, ha a két táblában van két azonos nevű oszlop, nem végezhetjük el. Ilyenkor az egyik oszlopot először át kell nevezni. Ezt a műveletet ritkán alkalmazzuk, mert nagyon ritkán van értelme, ezen kívül két nagyobb méretű tábla keresztszorzata egy hatalmas tárigényű, kezelhetetlen méretű táblázatot eredményezhet. A művelet:

relációnév1 TIMES relációnév2

Példa: Három fiú és három lány tanul táncolni. A táncórán minden lány bármelyik fiúval táncolhat, és fordítva. Az összes lehetséges kombinációt így állíthatjuk elő:
párok = lányok TIMES fiúk

Lányok		Fiúk	
l_név	l_magas	f_név	f_magas
Krisztina	180	Ákos	178
Ágnes	165	Balázs	182
Anikó	172	Ferenc	165

Párok			
l_név	l_magas	f_név	f_magas
Krisztina	180	Ákos	178
Krisztina	180	Balázs	182
Krisztina	180	Ferenc	165
Ágnes	165	Ákos	178
Ágnes	165	Balázs	182
Ágnes	165	Ferenc	165
Anikó	172	Ákos	178
Anikó	172	Balázs	182
Anikó	172	Ferenc	165

Megvalósítás Acces 2007-ben (választó lekérdezés, tervező nézet)

Párok

Lányok

*

l_név

l_magas

Fiúk

*

f_név

f_magas

Mező:	l_név	l_magas	f_név	f_magas
Tábla:	Lányok	Lányok	Fiúk	Fiúk
Rendezés:				
Megjelenítés:	☒	☒	☒	☒
Feltétel:				
vagy:				

Egyesítés (unió)

A halmazegyesítés kétoperandusú művelet. Mindkét táblának ugyanolyan szerkezetűnek kell lennie, hiszen az eredményreláció mindkét reláció rekord előfordulásait, azok összes sorát tartalmazza. Az eredményreláció struktúrája megegyezik a bemenő relációk struktúrájával. Azok a sorok amelyek mindkét táblában szerepelnek, az új táblában csak egyszer fognak megjelenni. A művelet:

relációnév1 UNION relációnév2

Példa: A „nyugati”, valamint „keleti” gépkocsik adatait tartalmazó táblából elkészítjük azt a táblát, ami valamennyi autó adatait tartalmazza:

ÚjAutó = Autó1 UNION Autó2

Autó1			
rendszám	típus	szín	ár
ATC-812	Opel	fehér	3 000 000
RSA-101	Opel	kék	2 800 000
LAK-941	Suzuki	fehér	1 900 000

Autó2			
rendszám	típus	szín	ár
ROC-512	Lada	piros	500 000
ALU-019	Trabant	zöld	30 000
SPA-310	Skoda	fehér	1 500 000

ÚjAutó			
rendszám	típus	szín	ár
ATC-812	Opel	fehér	3 000 000
RSA-101	Opel	kék	2 800 000
LAK-941	Suzuki	fehér	1 900 000
ROC-512	Lada	piros	500 000
ALU-019	Trabant	zöld	30 000
SPA-310	Skoda	fehér	1 500 000

Megvalósítás Access 2007-ben (összesítő lekérdezés, csak SQL nézetben lehetséges)

```
ÚjAutó
SELECT * FROM Autó1
UNION
SELECT * FROM Autó2;
```

Metszet

A metszet két relációban mindkét helyen előforduló rekord előfordulásokat adja vissza az eredménytáblában. A művelet csak abban az esetben hajtható végre, ha a két reláció szerkezete megegyezik. A művelet:

relációnév1 INTERSECTION relációnév2

Példa: Egy új táblába tesszük azokat az autókat, amelyek mindkét régi táblában szerepelnek:

KözösAutó = Autó1 INTERSECTION Autó2

Autó1			
rendszám	típus	szín	ár
ATC-812	Opel	fehér	3 000 000
ALU-019	Trabant	zöld	30 000
LAK-941	Suzuki	fehér	1 900 000

Autó2			
rendszám	típus	szín	ár
ROC-512	Lada	piros	500 000
ALU-019	Trabant	zöld	30 000
SPA-310	Skoda	fehér	1 500 000

KözösAutó			
rendszám	típus	szín	ár
ALU-019	Trabant	zöld	30 000

Megvalósítás Acces 2007-ben (választó lekérdezés, tervező nézetben)

KözösAutó

Autó1

*
rendszám
típus
szín
ár

Autó2

*
rendszám
típus
szín
ár

Mező:	rendszám	típus	szín	ár
Tábla:	Autó1	Autó1	Autó1	Autó1
Rendezés:				
Megjelenítés:	☒	☒	☒	☒
Feltétel:	[Autó2].[rendszám]	[Autó2].[típus]	[Autó2].[szín]	[Autó2].[ár]
vagy:				

Különbség

A különbség kétoperandusú művelet, és csak akkor hajtható végre, ha a táblázatok oszlopai megegyeznek. A művelet az elsőként vett relációban megtalálható, de a másodikban nem szereplő rekord előfordulásokat adja vissza az eredménytáblában. A különbség művelete nem szimmetrikus, azaz az eredmény függ az operandusok megadási sorrendjétől. A művelet:

relációnév1 EXCEPTION relációnév2

Példa: Az Opel autótípust már külön táblában tároljuk, ezért nincs szükség arra, hogy az összesített táblában is szerepeljenek az adatai:

NemOpel = Autó EXCEPTION Opel

rendszer	típus	szín	ár
ATC-812	Opel	fehér	3 000 000
RSA-101	Opel	kék	2 800 000
LAK-941	Suzuki	fehér	1 900 000
ROC-512	Lada	piros	500 000
ALU-019	Trabant	zöld	30 000
SPA-310	Skoda	fehér	1 500 000

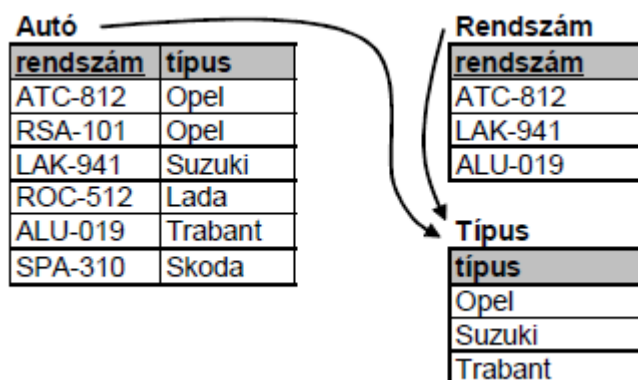
rendszer	típus	szín	ár
ATC-812	Opel	fehér	3 000 000
RSA-101	Opel	kék	2 800 000

rendszer	típus	szín	ár
LAK-941	Suzuki	fehér	1 900 000
ROC-512	Lada	piros	500 000
ALU-019	Trabant	zöld	30 000
SPA-310	Skoda	fehér	1 500 000

Osztás

Az osztás a gyakorlatban igen ritkán használt és kevés adatbázis-kezelő rendszerben megvalósított kétooperandusú művelet, mivel végrehajtása igen időigényes, s gyakorlati haszna sem jelentős. Egy R1 és R2 reláció hányadosa egy olyan reláció, melyben R1 mindazon rekordjainak projekciói tartoznak, melyek szorzata az R2-vel, legnagyobb részhalmazát alkotják az R1-nek.

Példa: Az Autó táblából kiválogatjuk a Rendszám táblában lévő rendszámokhoz tartozó autók típusait:



Ez egy összetett művelet, amit a következőképpen írhatunk le: Szelektáljuk az Autó táblát a Rendszám táblában megadott értékek szerint és végezzünk projekciót a Rendszámban nem szereplő mezőkre. A szelekciót a Rendszám tábla minden rekordjára el kell végezni, így annyi eredménytáblát kapunk, ahány sorból áll az osztó táblázat. Ezután venni kell az eredménytáblák metszetét, ami egyben az osztás eredményét tartalmazó tábla lesz.

Kiterjesztés

A művelet létrehozásának indíttatása az volt, hogy egyes lekérdezéseknél nem mindig csak a mezőértékekre, hanem az azokból **képzett** valamilyen **kifejezések** értékeire is kíváncsiak vagyunk.

Példa: A táblázatban tárolt adatok mellett szükségünk van a nettó árra is:

Autó				
rendszer	típus	szín	ár	ár*0,8
ATC-812	Opel	fehér	3 000 000	2 400 000
RSA-101	Opel	kék	2 800 000	2 240 000
LAK-941	Suzuki	fehér	1 900 000	1 520 000
ROC-512	Lada	piros	500 000	400 000
ALU-019	Trabant	zöld	30 000	24 000
SPA-310	Skoda	fehér	1 500 000	1 200 000

Csoportosítás

Bizonyos esetekben nem magára a konkrét rekord-előfordulásokra vagyunk kíváncsiak, hanem a rekord-előfordulások valamilyen összesítő adataira. A statisztikai adatok sokszor elegendő információt nyújtanak és jobban is kezelhetők, mint a részletes adatlisták.

Gyakran alkalmazott összesítések:

- count – az előfordulások darabszáma,
- sum – előfordulások valamely mezőjének összege,
- max – előfordulások valamely mezőjének maximuma,
- min – előfordulások valamely mezőjének minimuma,
- avg – előfordulások valamely mezőjének átlaga.

Példa: Az alábbi táblából összeszámoljuk, hogy hány darab autónk van, és mennyi az autók átlagára:

Autó			
rendszer	típus	szín	ár
ATC-812	Opel	fehér	3 000 000
RSA-101	Opel	kék	2 800 000
LAK-941	Suzuki	fehér	1 900 000
ROC-512	Lada	piros	500 000
ALU-019	Trabant	zöld	30 000
SPA-310	Skoda	fehér	1 500 000

COUNT(típus)
6

AVG(ár)
1 621 667

Források

Tringer Éva - Fodor Ildikó:

ECDL 5. modul: Adatbázis-kezelés

Kossuth Kiadó 1999.

Fred Rolland:

Adatbázis-rendszerek

Panem Könyvkiadó Kft. Budapest, 2002.

Hampel György:

Adatbázisok

Szegedi Tudományegyetem Szegedi Élelmiszeripari Főiskolai Kar 2003.

Szelezsán János:

Adatbázisok

LSI Informatikai Oktatóközpont Budapest, 2004.

Bártfai Barnabás:

Access 2007 zsebkönyv

BBS-INFO Kiadó, 2007.

dr. Katona Endre:

Adatbázisok - Előadási jegyzet (BSc)

Szegedi Tudományegyetem Informatikai Tanszékcsoport 2010.